



Máquinas de Estado

INFO II

Profesor: Ing. Federico Weber

TEMAS

- ¿Qué son?
- ¿Por qué usarlas?
- Máquinas de Mealy y Moore
- Implementación de una máquina de estados en C
- Ejemplo 1: Contador de cantidad de 1 impares
- Ejemplo 2 y 3: Detector ABCD (+ contador)
- EFSM
- Ejemplo 4: Tratamiento de protocolo de comunicaciones
- TFSM: Concepto y ejemplo con un semáforo simple
- Ejemplo 5: Implementación de una EFSM con puntero a función

Timed FSM

- Hasta acá la transición la disparaba una ENTRADA (una tecla, un byte).
- A veces la transición la dispara el TIEMPO, no una entrada.
- Timed FSM (TFSM): una FSM donde algunas transiciones ocurren por un reloj o una demora. **Una TFSM no es más que una EFSM con un contador de tiempo tratado como caso especial.**

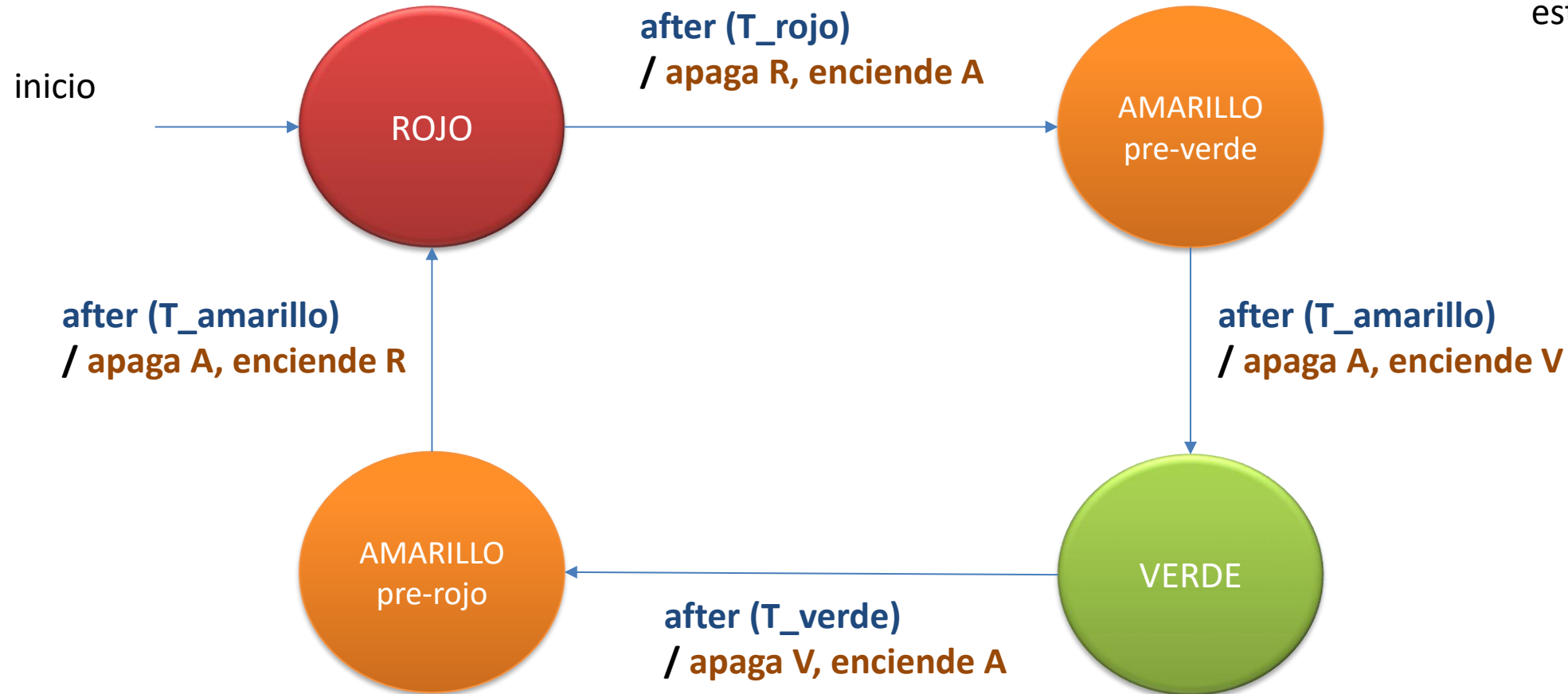
Con la notación que ya vimos, el tiempo entra como una extensión natural. Dos formas equivalentes de escribir "esperá 5 segundos y transicióná":

- **Como evento:** `after(5s) / acción` — evento sintético que "dispara" cuando pasan 5 segundos desde que entramos al estado.
- **Como guarda:** `[t ≥ 5s] / acción` — donde `t` es un reloj implícito que se **resetea al entrar** al estado.

Ejemplo SEMAFORO

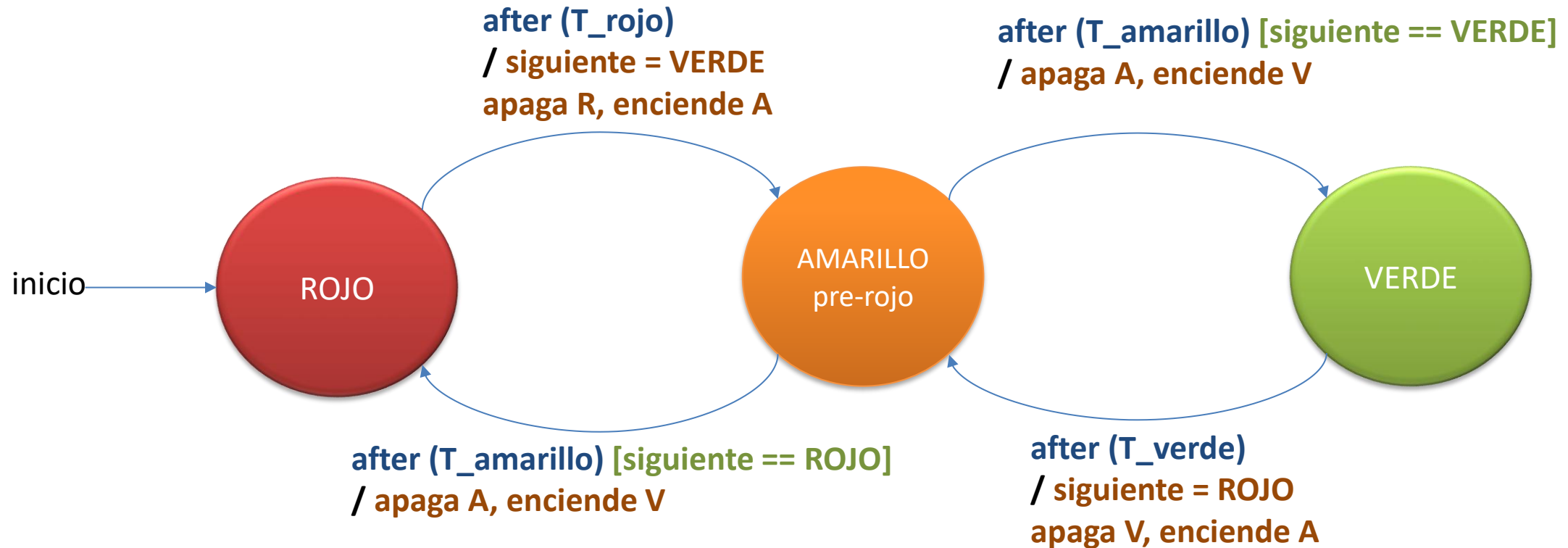
¿Mealy o Moore?

¿Lo podemos llevar a 3 estados?



•Formato: **evento** [guarda] / **acción**

Ejemplo SEMAFORO



•Formato: **evento** [guarda] / acción

Limitaciones de FSM implementadas con switch-case:

- 1. No escala bien: En máquinas de estados grandes, el código puede volverse muy extenso y complicado debido a la lógica condicional anidada.
- 2. Duplicación de código: en switch/case no podés probar un estado sin ejecutar toda la máquina; con punteros a función cada estado es una función independiente que podés testear por separado y hasta poner en su propio archivo.
- 3. No se divide bien: el código que maneja los estados y el código que realiza acciones específicas en respuesta a eventos se mezclan. Esto significa que no está claro qué parte del código se encarga de la transición entre estados y qué parte maneja las acciones que deben realizarse en esos estados.

Implementación de una maquina de estados en C – caso punteros a función

```
// 1. Definimos el tipo de función para los estados
typedef void (*StateFunc)(); // Puede adaptarse si necesitás parámetros

// 2. Prototipos de las funciones que representan los estados
void state_0();
void state_1();
void state_2();
// ...
void state_n();

// 3. Variable que apunta al estado actual
StateFunc currentState;

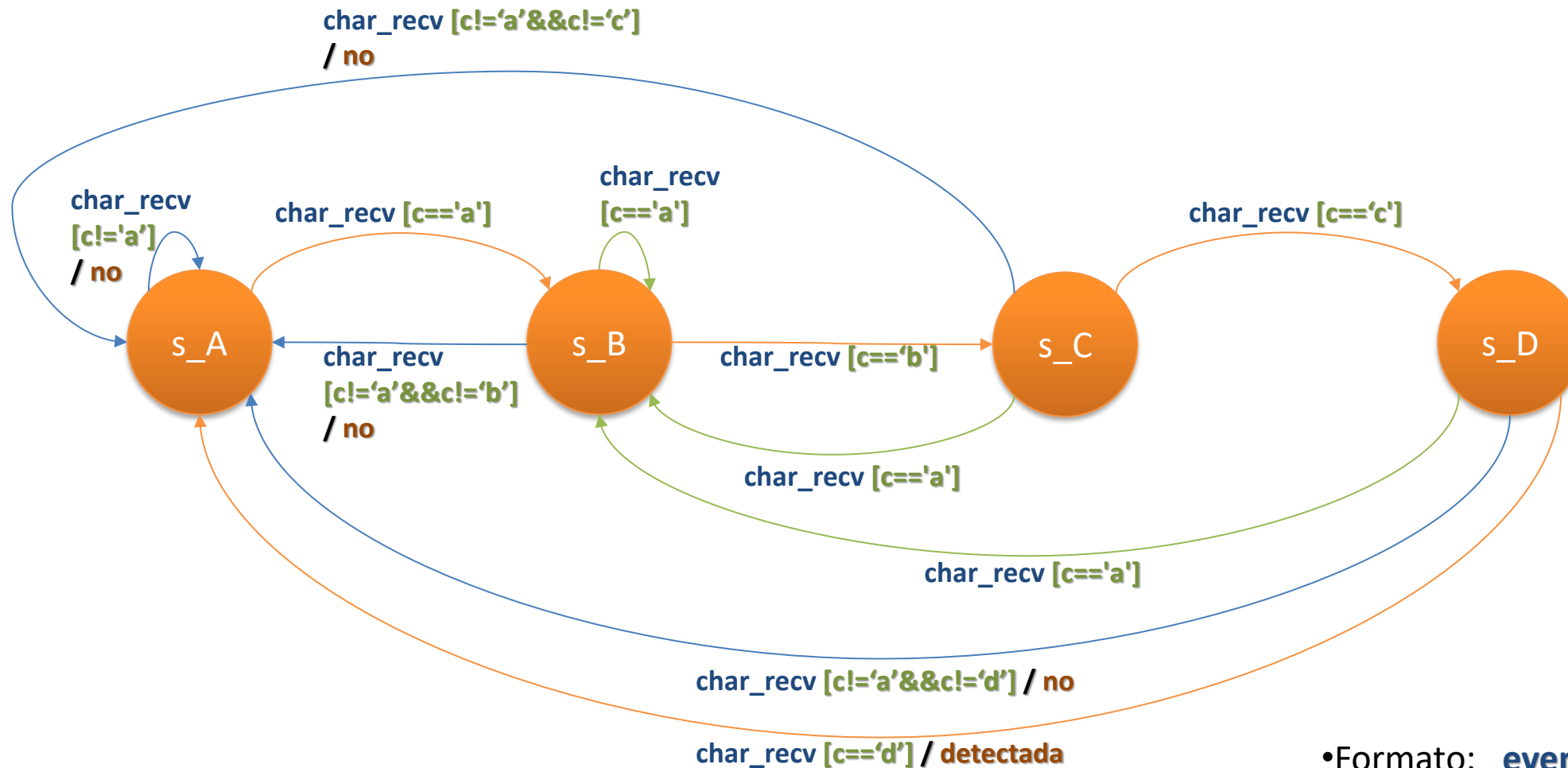
int main(void) {
    // 4. Estado inicial
    currentState = state_0;

    while ("condición de salida") {
        currentState(); // Ejecutar estado actual
    }
}
```

- Fácil de escalar: solo se agregan funciones
- Ideal para FSM con muchos estados o transiciones complejas

Ejemplo 5

- Realizar un programa que detecte la secuencia de caracteres “abcd” ingresadas por teclado. Implementar con punteros a función. Tarea de la clase pasada.



•Formato: **evento** [guarda] / acción

Ejercicio 1 – El mas clásico de todos

- Máquina expendedora de golosinas: vende chicles a 0.15\$ y chocolates a 0.2\$. La máquina acepta monedas de 5 y 10 centavos. Cada vez que se introduce una moneda, la máquina emite un sonido (click), si dicha moneda es de 5 centavos o, dos (dos clicks) si es de 10 centavos. Se tienen 2 botones selectores para elegir la golosina deseada y uno de devolución que permite recuperar el dinero ingresado sino se efectuó la entrega de ninguna golosina.
Realizar la maquina de estados.

Ejercicio 2

- En la intersección de dos avenidas se instala un semáforo con el siguiente comportamiento.

sem1	R	R	A	V	A	R
sem2	V	A	R	R	R	A
tiempo	30s	2s	2s	30s	2s	2s

- Realizar una maquina de estados que controle ambos semaforos.



FIN